



Android Data Binding: Write Less to do More



The Android data binding framework significantly reduces the code that needs to be written. This second article in the series covers the aspects of two-way binding, expression chaining, implied event updates and Lambda expressions.

We covered one method of data binding in the first part of this article (*OSFY*, July 2016). Along with that, we also looked at certain basics like how to get data binding up and running in Android Studio, and the use of binding adapters to add our own attributes to layouts easily. While one-way data binding enables us to get our data from the model to the UI, there are times when we need to get input from the UI and send it to our model. This is when two-way data binding comes to our rescue. It helps the data flow in both directions in our application, i.e., from the model to the UI and vice versa.

When Android data binding supported only one-way binding, people used to achieve two-way data binding by including callback methods on the models, and then calling those methods from XML by using listeners such as *TextWatchers*. But now, as the data binding framework on Android supports two-way data binding, it becomes very easy to bind an attribute to a field with two-way binding. Let's take a look at an example that will make this concept very clear.

Previously, when we bound an attribute to a field, we used the following syntax:

```
android:text="@{model.text}"
```

This will essentially declare that the field is a one-way data binding field. Now, to make the field bind both ways, we just need to replace `@` with `@=`, as follows:

```
android:text="@={model.text}"
```

And that's pretty much it. This field is now two-way data bound with the model, i.e., if data in the variable changes, it'll update the UI and when the UI changes the data, it'll update the model. But, of course, the field needs to be observable, so the rules and syntax we covered in the previous article on this topic still apply.

Most of the time, when using two-way data binding, only adding the 'equals' sign will make the field two-way bound. But there may be cases when the data generated by the UI is not in the form that the model accepts. To help with such cases, we have *InverseBindingAdapters*, which are very useful to get data from the UI and convert it into a form that the model expects. *InverseBindingAdapters* look and behave almost the same way as *BindingAdapters* do. Let's explore the

example we described above:

```
android:text="@={model.text}"
```

Now the variable `text` is a string type variable and the attribute `android:text` or method `setText` expects a string, so it works perfectly in one-way data binding, but when we want to pass the value of the text to the model, we need to get the data from the view. To get the text from the view, we will call `getText` on the view, but `getText` doesn't return a string. It returns a *CharSequence* in case of *TextView* and it returns an editable in the case of *EditText*. The conversion from a *CharSequence* or an editable to a string before setting it on a variable is done in the *InverseBindingAdapter*, which expects a single parameter—the view on which the attribute is and the return type of the method should be the type which the model expects (in our case, string).

It is important to check the return value of the method carefully, because the method will be called based on the return type of the method. Now, to give our problem a solution, we write an *InverseBindingAdapter* as shown below:

```
@InverseBindingAdapter(attribute = "android:text")
public static String getStringFromTextView(TextView textView)
{
    return textView.getText().toString();
}
```

The attribute parameter inside the annotation is the attribute you want applied on this adapter. That's a simple and basic introduction to how two-way data binding can be achieved with the Android data binding framework.

Expression chaining

It often happens that more than one view in our layout hierarchy depends on a single variable or a single expression. So what we need to do is repeat all that code in all the views. Data binding has now introduced expression chaining, in which you can put that condition in only one of the views and then depend on that view's state in other views. Let's see what it looks like in code.

Assuming that you want to toggle some view's visibility based on the state of a *CheckBox*, write the code only once and refer to the view's visibility in other views by using that view's ID as shown in the following code: